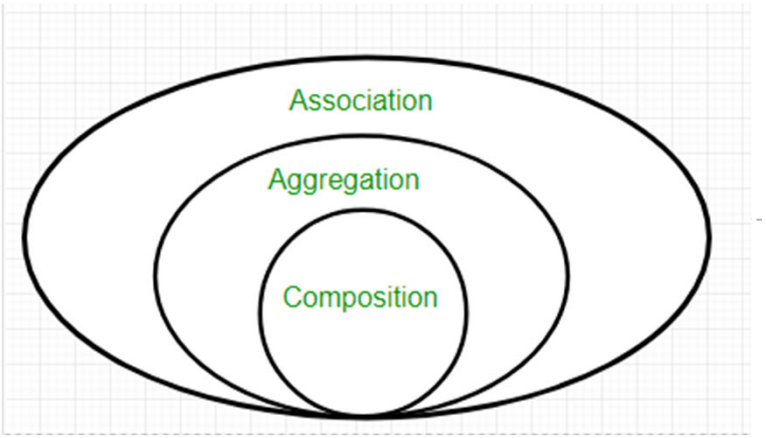
CompSci 230
Object Oriented Software Development

Aggregation & Composition

Agenda & Reading

- Topics:
  - Introduction
  - Aggregation
  - Composition
- Reading
  - Java how to program Late objects version (D & D)
    - Chapter 7
  - The Java Tutorial
    - Classes:
      - http://docs.oracle.com/javase/tutorial/java/javaOO/classes.html
    - Objects
      - http://docs.oracle.com/javase/tutorial/java/javaOO/objects.html

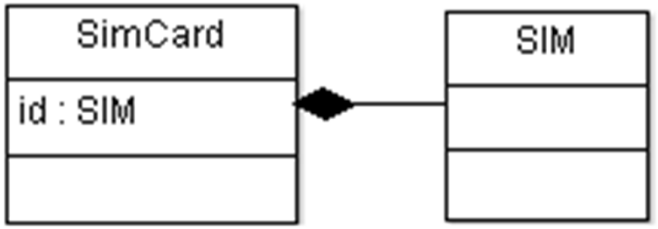
Relationships



- There are three kinds of relationships.
  - Association & Dependency
    - A relationship between instances of the two classes. There is an association between two classes if an instance of one class must know about the other in order to perform its work.
    - Association can be one-to-one, one-to-many, many-to-one, many-to-many
  - Aggregation (has-a relationship) & Composition
    - An association in which one class belongs to a collection.
      - Normal Aggregation
      - Composition (strong aggregation), a Part cannot exist once Whole is destroyed
  - Generalization (is-a relationship)
    - An inheritance link indicating one class is a superclass of the other.

Composition and Aggregation

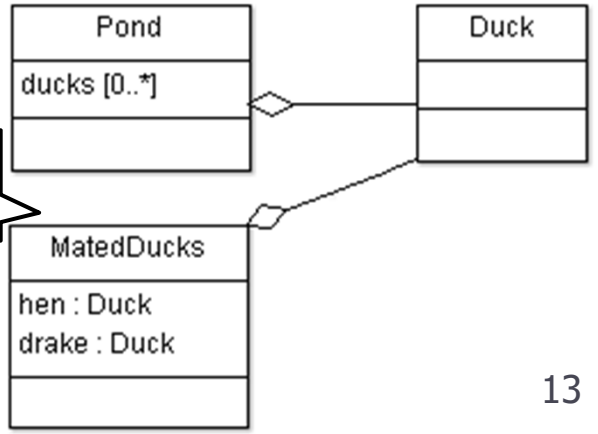
- These relationships create a complex class from one or more existing classes (or instances), in a whole/part relationship.
- Composition ("owns-a", "has-a"):
  - An object is a component of at most one composition.
  - When a composition instance is destroyed, all objects belonging to this instance are destroyed as well.
  - Examples:
    - SimCard has-a SIM.
    - Human has a Heart.
    - Car has an engine



one class includes another class and is dependent on it so that it cannot functionally exist without another class.

- Aggregation ("has-a"):
  - An object can be in many aggregations.
  - Example: **`Pond`** has-a **`Duck`**

An aggregation container class and referenced class can have an independent existence.



1.Aggregation

- In Java, aggregation represents HAS-A relationship, which means when a class contains reference of another class
  - i.e. This means a class **contains an object of another class**.
  - is used to refer one way relationship between two objects. For examples
    - Student class can have reference of Address class but vice versa does not make sense.
    - An employee has a home address. But vice versa, the home address has employees, doesn't sound right.

- Advantage:
  - maintain code re-usability. If an entity has a relation with some other entity than it can reuse code just by referring that.

1.Aggregation
Date

- Class Date
  - Instance variables: day, month and year to represent a date
  - The constructor receives three int parameters. It also validate day if it's out of range or invalid
  - The toString method return the object's string representation.

```
class Date {
   private int month; // 1-12
   private int day;   // 1-31 based on month
   private int year;  // any year
   public Date( int theMonth, int theDay, int theYear ) {
      month = checkMonth( theMonth ); // validate month
      year = theYear; // could validate year
      day = checkDay( theDay ); // validate day
   ...
   public String toString() {
      return String.format( "%d/%d/%d", month, day, year );
   }
...
```

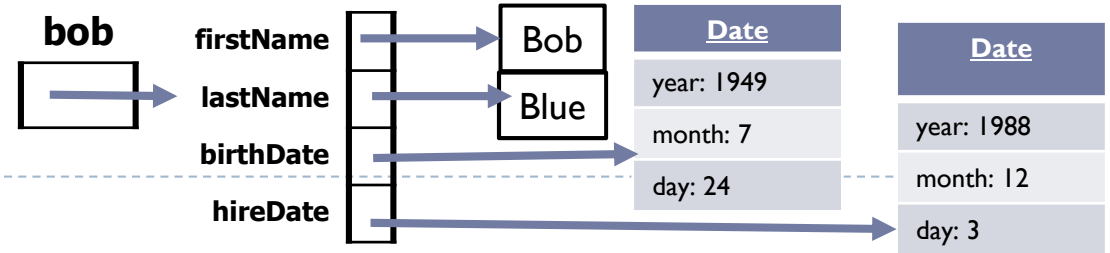
EmployeeTest.java

1.Aggregation

- A class can have references to objects of other classes as members.
- This is a **has-a relationship**.
- Example: Employee and hire date (Date)
- Class Employee
  - Instance variables: firstName, lastName, birthDate and hireDate
  - Members firstName and lastName are references to String objects
  - Members birthDate and hireDate are references to Date objects

```
Date birth = new Date(7, 24, 1949);
Date hire = new Date(3,12,1988);
Employee bob = new Employee("Bob", "Blue", birth, hire);
System.out.println(bob);
```

```
Blue, Bob; Hired: 3/12/1988 Birthday: 7/24/1949
```



1.Aggregation Employee

- Employee & Date : has-a relationship
  - Employee Class
  - Date class

```
public class Employee {
   private String firstName;
   private String lastName;
   private Date birthDate;
   private Date hireDate;
  ...
```

```
class Date {
  private int month; // 1-12
  private int day;   // 1-31 based on month
  private int year;
  public Date( int theMonth, int theDay,
    int theYear ){
  ...
}
```

```
public Employee(String first, String last,
  Date birth, Date hire ) {
    firstName = first;
    lastName = last;
    this.birthDate = birth;
    hireDate = hire;
}
```

1.Aggregation
Java Source Files

- When you compile a .java file containing more than one class, the compiler produces a **separate class** file with the .class extension for every compiled class.
- When one source-code (.java) file contains multiple class declarations, the compiler places both class files for those classes in the same directory.
- A source-code file can contain only **one public class**—otherwise, a compilation error occurs.

EmployeeTest.java

```
class Date {
...
}
class Employee {
   ...
}
public class EmployeeTest {
  public static void main( String args[] ) {
  ...
}
```

Exercise 1-2: SimpleRectangle & SimpleTriangle

- Define a class named SimpleRectangle to represent a rectangle.

```
SimpleRectangle p = new SimpleRectangle();
System.out.println(p);
SimpleRectangle p2 = new SimpleRectangle(10, 5);
System.out.println(p2);
```

```
Rectangle, area=1.00
Rectangle, area=50.0
```

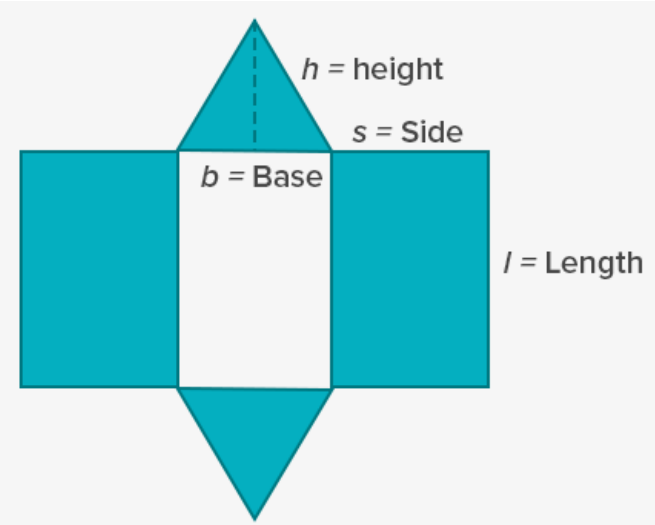
- Define a class named SimpleTriangle to represent a triangle.

```
SimpleTriangle p = new SimpleTriangle();
System.out.println(p);
SimpleTriangle p2 = new SimpleTriangle(10, 5);
System.out.println(p2);
```

```
Triangle, area=0.50
Triangle, area=25.00
```

Exercise 3

- Define a class named TriangularPrism to represent a triangular prism. A triangular prism consists of two triangles, two side rectangles and one bottom rectangle.
  - private SimpleTriangle triangle
  - private SimpleRectangle sideRect, bottomRect
  - constructor:
    - triangleBase, triangleHeight, triangleSide, length
    - triangle: triangleHeight & triangleBase
    - sideRect: triangleSide, length
    - bottomRect: length, triangleBase
  - Surface area:
    - area of sideRect x 2 + area of triangle x 2 + area of bottomRect




```
TriangularPrism p = new TriangularPrism(6, 4, 5, 12);
System.out.println(p);
```

```
Prism, Surface area=216.00
```

Exercise 4-5

- Department class has-a relationship with Student class.
  - Define a class named Student which represents a student.
    - name, studentId

```
Student p1 = new Student("Mia", 123);
System.out.println(p1);
Student p2 = new Student("Michael", 456);
System.out.println(p2);
```

```
Mia(123)
Michael(456)
```

  - Define a class named Department which represents a school department
    - name, students (ArrayList<Student>)
    - addStudent(Student s)

```
Department("Computer Science");
cs.addStudent(p1);
cs.addStudent(p2);
System.out.println(cs);
```

```
Computer Science(number of students=2)
```

- Some more examples:
  - Book -> author, Book-> publisher
  - Student -> address

2.Composition

- Composition is a more restricted form of Aggregation.
- Composition can be described as when one class which includes another class, is dependent on it in such a way that it cannot functionally exist without the class which is included.
  - For example a class Car cannot exist without Engine, as it won't be functional anymore.
    - i.e. an engine cannot exist independently without a car.
  - For example, an Employee has a bank account.
    - A bank account cannot exist without an account holder.

- It is a "part-of-a-whole" relationship in which the **part** does not exist without the **whole**.
  - hence when the containing object is destroyed; the other object is also destroyed
  - The whole has a stronger relationship with the part.

2.Composition Car

```
Car c1 = new Car(30, "red", 45);
System.out.println(c1);
```

```
Car(cylinder capacity:30ml,color=red,max speed=45)
```

- A class Car cannot exist without Engine.
  - Class CarEngine
    - Instance variables: cylinderCapacity

```
class CarEngine {
  private int cylinderCapacity;
  public CarEngine(int cap) { cylinderCapacity = cap; }
  public String toString() {
    ...
```

  - Class Car
    - instance variables: engine, color, maxSpeed

```
class Car {
  private String color;
  private int maxSpeed;
  private CarEngine engine;
  public Car(int capacity, String color, int maxSpeed){
    this.engine = new CarEngine(capacity);
    this.color = color;
    this.maxSpeed = maxSpeed;
```

2.Composition Exercise 6

CodeRunner

29	28	27
16	15	14
3	2	1

- A 3x3 matrix is represented by the following array:
  - { {29, 28, 27}, {16, 15, 14}, {3, 2, 1} };
- Complete the following MyMatrix class

```
class MyMatrix {
  private int[][] data;
  private final static int  SIZE = 3;
  public MyMatrix() {  //complete this }
  public MyMatrix(int x11, int x12, int x13, int x21, int x22, ...
```

```
public String toString() {
    StringBuffer sb = new StringBuffer("");

     return sb.toString();
}
```

```
MyMatrix myMatrix2  = new MyMatrix(1, 2, 3, 4, 5, 6, 7, 8, 9);
System.out.println(myMatrix2 );
```

```
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
```

2.Composition Exercise 7

- Complete the add method
  - returns a new MyMatrix object.

```
public MyMatrix add(MyMatrix other) {
  MyMatrix result = new MyMatrix();









  return result;
}
```

```
[30, 30, 30]
[20, 20, 20]
[10, 10, 10]
```

```
MyMatrix myMatrix2  = new MyMatrix(1, 2, 3, 4, 5, 6, 7, 8, 9);
MyMatrix myMatrix1  = new MyMatrix(29, 28, 27, 16, 15, 14, 3, 2, 1);
MyMatrix result = myMatrix1.add(myMatrix2);
System.out.println(result);
```

2.Composition Exercise 8

CodeRunner

- Complete the subtract method.
  - returns a new MyMatrix object.

```
public MyMatrix subtract(MyMatrix other) {
  MyMatrix result = new MyMatrix();









  return result;
}
```

```
[28, 26, 24]
[12, 10, 8]
[-4, -6, -8]
```

```
MyMatrix myMatrix2  = new MyMatrix(1, 2, 3, 4, 5, 6, 7, 8, 9);
MyMatrix myMatrix1  = new MyMatrix(29, 28, 27, 16, 15, 14, 3, 2, 1);
MyMatrix result = myMatrix1.subtract(myMatrix2);
System.out.println(result);
```

Exercise 9

- Complete the multiply method.
  - returns a new MyMatrix object.

```
public MyMatrix multiply(MyMatrix other) {
  MyMatrix result = new MyMatrix();



  return result;
}
```

$$AB = \begin{bmatrix} 2 & 0 & -1 \\ 3 & 5 & 2 \\ -4 & 1 & 4 \end{bmatrix} \begin{bmatrix} 5 & 1 & -2 \\ -1 & 0 & 4 \\ 2 & -3 & 3 \end{bmatrix}$$

$$= \begin{bmatrix} 2\times 5+0\times(-1)+(-1)\times 2 & 2\times 1+0\times 0+(-1)\times(-3) & 2\times(-2)+0\times 4+(-1)\times 3 \\ 3\times 5+5\times(-1)+2\times 2 & 3\times 1+5\times 0+2\times(-3) & 3\times(-2)+5\times 4+2\times 3 \\ (-4)\times 5+1\times(-1)+4\times 2 & (-4)\times 1+1\times 0+4\times(-3) & (-4)\times(-2)+1\times 4+4\times 3 \end{bmatrix}$$

$$= \begin{bmatrix} 8 & 5 & -7 \\ 14 & -3 & 20 \\ -13 & -16 & 24 \end{bmatrix}$$

Exercise 10

- Complete the scalar_multiply() method.
  - Each entry in the matrix is multiplied by the given scalar value.
  - Note:
    - makes changes in place
    - should not return anything

```
public void scalar_multiply(int value) {

}
```

```
[10, 20, 30]
[40, 50, 60]
[70, 80, 90]
```

```
MyMatrix p  = new MyMatrix(1, 2, 3, 4, 5, 6, 7, 8, 9);
p.scalar_multiply(10);
System.out.println(p);
```

Aggregation Vs Composition

Aggregation allows child objects or contained objects to exist independently. **For example,** in a relationship where School has Employees, if we delete School, Employees will remain and can function on its own.	In composition, the contained object cannot exist independently. **For example,** a car has Engine. If we delete Car, Engine class cannot function on its own.
Aggregation is a 'has-a' relationship.	The composition is a form of 'has-a' relationship but viewed as part-of-a-whole' relation.
The aggregation has one to one association only.	Composition allows other relationships provided in the association.
The aggregation has a weak association between objects.	The composition has a strong association between objects.